

Managing Network Access with Authentication, Authorization, and Accounting

Related CCNA v1.1 exam topic:

- 2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, TACACS+/RADIUS, and cloud managed)
- 5.3 Configure and verify device access control using local passwords
- 5.8 Compare authentication, authorization, and accounting concepts

In this lab, you will explore how to:

- Configure a default authentication and authorization method
- Configure a named authentication and authorization method
- Configure server-based AAA using TACACS+
- Configure server-based AAA using RADIUS

Setup and Scenario

In this set of lab-based demonstrations, you are a network engineer tasked with exploring different ways that AAA can be enabled on a Cisco router.

Knowledge Check

In networking, AAA stands for:

- Authentication
- Authorization
- Accounting

AAA is a framework for controlling access to network resources:

- Authentication: Verifies who you are (username/password, certificates, biometrics)
- Authorization: Determines what you're allowed to do (access permissions, resource limits)
- Accounting: Tracks what you did (logging usage, session duration, data transferred for billing/auditing)

Be sure to **START** the lab before continuing

Note: The credentials for all devices are **cisco / cisco**

Part 1: Configure a default authentication and authorization method

Before we start testing RADIUS and TACACS+ AAA services later in the lab, let's setup a default authentication and authorization method for R1's console access. This method will only use the enable secret password for access to the CLI. By doing this, we can ensure we won't lose access to the device if something goes wrong with our AAA configuration in Part 3 and Part 4.

Step 1

By default, AAA is not enabled. Start by enabling AAA functions on the router.

```
R1# configure terminal
R1(config)# aaa new-model
```

When enabling AAA globally, the routers's local login passwords are disabled, if any were configured.

Step 2

A single named method list allows you to configure up to four AAA methods for a given operation on the device. These methods will be used in the order they are configured. If any method results in a rejection, then the attempt fails. If the method is unavailable, then the next configured method will be used.

Method lists are named with either a custom name or the name `default`. A custom-named method list must be explicitly configured. A default method list applies to all device lines (console, VTY, AUX), without any additional configuration.

Let's start by creating a default method list for login that uses the enable password.

```
R1(config)# aaa authentication login default enable
```

Because this method list is the default for the login process, it is now in effect for every possible type of login, without any additional configuration. Issue the `show run | section line con` command and note that the `login` command is not present. Instead, the default method will "intercept" login attempts.

```
R1# show run | section line con
line con 0
  exec-timeout 0 0
  logging synchronous
R1#
```

Completely log out of the console session, and then try to log back in. You should be prompted for a password only. Enter the enable secret password, which is configured as `cisco`.

```
R1(config)# exit
R1# exit
```

R1 con0 is now available

Press RETURN to get started.

User Access Verification

Password: *(cisco)*

```
R1> enable
Password: (cisco)
R1
```

Step 3

AAA authorization is disabled on the console by default. We can enable it manually but you should be careful because a misconfiguration could lock you out of the device entirely through the console port.

```
R1(config)# aaa authorization console
```

TIP: In a production network, you can use the `reload in` command to ensure that any AAA configuration errors don't result in locking you out of the device.

Step 4

Next we will enable authorization for EXEC shell access, using the default method list to authorize users for privileged EXEC mode access. This configuration enforces that after a user is authenticated they will have permission to enter privileged EXEC mode based solely on the enable secret password.

The command `aaa authorization exec default if-authenticated` configures a network device to allow any user access to EXEC shell commands, provided they have already been successfully authenticated. This command simplifies authorization by relying on successful login, which means the device doesn't need to perform a separate, detailed authorization check for EXEC sessions

```
R1(config)# aaa authorization exec default if-authenticated
```

Step 5

Let's verify the complete config for Part 1.

```
R1# show run | section aaa
aaa new-model
aaa authentication login default enable
aaa authorization console
aaa authorization exec default if-authenticated
aaa session-id common
```

Let's test if our initial AAA config for the default method is working. First, enable console logging and then enable AAA debugging for authentication and authorization. You will then completely log out of the console and log back in.

```
R1(config)# logging console
R1(config)# exit
R1# debug aaa authentication
AAA Authentication debugging is on
R1# debug aaa authorization
AAA Authorization debugging is on
R1# exit
```

R1 con0 is now available

Press RETURN to get started.

```
*Nov 21 04:08:52.712: %SYS-5-CONFIG_I: Configured from console by console
```

User Access Verification

Password: *(cisco)*

```
*Nov 21 04:08:55.100: AAA/BIND(00000011): Bind i/f
*Nov 21 04:08:55.100: AAA/AUTHEN/LOGIN (00000011): Pick method list 'default'
*Nov 21 04:08:55.100: AAA/AUTHEN/ENABLE(00000011): Processing request action LOGIN
*Nov 21 04:08:55.100: AAA/AUTHEN/ENABLE(00000011): Done status GET_PASSWORD
```

R1>

```
*Nov 21 04:08:57.529: AAA/AUTHEN/ENABLE(00000011): Processing request action LOGIN
*Nov 21 04:08:57.578: AAA/AUTHEN/ENABLE(00000011): Done status PASS
*Nov 21 04:08:57.578: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: ] [Source: LOCAL] [localport: 0] at 04:08:57 UTC Fri Nov 21 2025
R1>en
```

Password: *(cisco)*

```
*Nov 21 04:08:57.578: AAA/AUTHOR (00000011): Method=If-authen for method list id=00000000Skip author
*Nov 21 04:08:58.934: AAA: parse name=tty0 idb type=-1 tty=-1
*Nov 21 04:08:58.934: AAA: name=tty0 flags=0x11 type=4 shelf=0 slot=0 adapter=0 port=0 channel=0
*Nov 21 04:08:58.934: AAA/MEMORY: create_user (0x700B65680D00) user='NULL' ruser='NULL' ds0=0 port='tty0' rem_addr='async' authen_type=ASCII s
*Nov 21 04:08:58.934: AAA/AUTHEN/START (297861893): port='tty0' list='' action=LOGIN service=ENABLE
*Nov 21 04:08:58.934: AAA/AUTHEN/START (297861893): console enable - default to enable password (if any)
*Nov 21 04:08:58.934: AAA/AUTHEN/START (297861893): Method=ENABLE
*Nov 21 04:08:58.934: AAA/AUTHEN (297861893): status = GETPASS
```

R1#

```
*Nov 21 04:09:00.239: AAA/AUTHEN/CONT (297861893): continue_login (user='(undef)')
*Nov 21 04:09:00.239: AAA/AUTHEN (297861893): status = GETPASS
*Nov 21 04:09:00.239: AAA/AUTHEN/CONT (297861893): Method=ENABLE
*Nov 21 04:09:00.284: AAA/AUTHEN (297861893): status = PASS
*Nov 21 04:09:00.284: AAA/MEMORY: free_user (0x700B65680D00) user='NULL' ruser='NULL' port='tty0' rem_addr='async' authen_type=ASCII service=EL
R1#
```

You can see in the output that there were two authentication processes (one for user EXEC and one for privileged EXEC) and that the authorization process was skipped since we were successfully authenticated.

Part 2: Configure a named authentication and authorization method

In Part 2, we will configure a named method and use it for VTY access instead of relying on the default method configured in Part 1. We will call this method REMOTE-ACCESS and we will configure it to use a local username database for authentication and authorization.

Step 1

First, let's configure a local user named `admin` with a secret password of `adminpass`.

```
R1(config)# username admin secret adminpass
```

Step 2

Next, setup the named method for both authentication and authorization to use the local database only.

```
R1(config)# aaa authentication login REMOTE-ACCESS local
R1(config)# aaa authorization exec REMOTE-ACCESS local
```

If the previous `debug` commands are still active, you will see syslog messages in response to you adding these two commands.

Step 3

Next enable authentication and authorization on the VTY lines to use the named method list REMOTE-ACCESS instead of the previously configured default method.

```
R1(config)# line vty 0 4
R1(config-line)# login authentication REMOTE-ACCESS
R1(config-line)# authorization exec REMOTE-ACCESS
```

Step 4

To test that your remote access is configured correctly, open a console connection to PC1 and establish an SSH session to R1 using the local database as login credentials.

Enter **yes** when asked about accepting the RSA key.

```
PC1:~$ ssh admin@10.1.1.1
The authenticity of host '10.1.1.1 (10.1.1.1)' can't be established.
RSA key fingerprint is SHA256:z4NlXzDZisKi7GI5sClcIQ20v50lYdK9aK+73ZJyRaA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.1.1.1' (RSA) to the list of known hosts.
(admin@10.1.1.1)
Password: (adminpass)
```

```
R1>
```

Notice that the admin user was granted access to user EXEC mode.

If you return to the R1 console and still have the debug commands enabled, you should see the following syslog messages:

```
*Nov 21 22:55:59.947: %SSH-5-SSH2_SESSION: SSH2 Session request from 10.1.1.10 (tty = 0) using crypto cipher 'chacha20-poly1305@openssh.com', 1
R1(config-line)#
*Nov 21 22:56:00.150: AAA/BIND(00000012): Bind i/f
*Nov 21 22:56:00.150: AAA/AUTHEN/LOGIN (00000012): Pick method list 'REMOTE-ACCESS'
R1(config-line)#
*Nov 21 22:56:02.751: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: admin] [Source: 10.1.1.10] [localport: 22] at 22:56:02 UTC Fri Nov 21 2
*Nov 21 22:56:02.751: %SSH-5-SSH2_USERAUTH: User 'admin' authentication for SSH2 Session from 10.1.1.10 (tty = 0) using crypto cipher 'chacha2
R1(config-line)#
*Nov 21 22:56:02.753: AAA/AUTHOR (0x12): Pick method list 'REMOTE-ACCESS'
*Nov 21 22:56:02.753: AAA/AUTHOR/EXEC(00000012): processing AV cmd=
*Nov 21 22:56:02.753: AAA/AUTHOR/EXEC(00000012): processing AV priv-lvl=1
*Nov 21 22:56:02.753: AAA/AUTHOR/EXEC(00000012): Authorization successful
```

The output above confirms the following:

- An SSH session was established from 10.1.1.10
- The REMOTE-ACCESS method was selected for authentication
- Login was established using the **admin** username
- Access to priv-lvl 1 (user EXEC) was authorized

Why was the admin user only granted a **priv-lvl** of 1? Let's see:

```
R1# show run all | s username
username admin privilege 1 secret 9 $9$8lOCJ4JfIfaDFU$avS7RHOkkrHABVZwen0da0ACKvzvQLp2AS142TzfIEI
R1#
```

Notice that by default, when we entered the **username** command, our admin user received a privileged level of 1.

Perhaps for an admin user we would prefer that they have direct access to privileged EXEC mode when logging into the VTY lines. To accomplish this, we need to modify the local database configuration. We will re-enter the username command but this time we will specify a privileged EXEC level of 15 for the admin user.

First, disconnect the SSH session from PC1 before continuing. Then enter the following command on R1 to override the current admin user configuration:

```
R1(config)# username admin privilege 15 algorithm-type scrypt secret adminpass
R1(config)#
*Nov 21 23:05:59.653: %AAA-6-USER_PRIVILEGE_UPDATE: username: admin privilege updated with priv-15
```

Again, notice the AAA syslog message when you modify the privilege level of the user.

From PC1, establish a new SSH session with R1 to test the increase in privilege.

```
PC1:~$ ssh admin@10.1.1.1
(admin@10.1.1.1) Password: (cisco)
```

R1#

The prompt shows that the admin user now has direct access to privileged EXEC mode when establishing an SSH session with R1.

If you return once again to the R1 console and you should see a syslog message confirming that level 15 access was authorized:

```
*Nov 22 02:07:52.351: AAA/AUTHOR/EXEC(00000013): processing AV priv-lvl=15
```

Disconnect the SSH connection to R1 before continuing to Part 3 of the lab.

Part 3: Configure server-based AAA using TACACS+

So far, we've configured authentication and authorization on R1 for console and VTY access. But the local authentication and authorization used for VTY access is not very scalable, because the configuration has to be repeated on all network devices and any changes to the AAA policy must then be updated individually on all network devices. Using a centralized AAA server is much more scalable. Policies are defined centrally on the server. Updates that are made to the centralized policy are available to all network devices. Also, using centralized AAA servers supports the use of the third "A": accounting.

In the topology, the TACACS+ server is a container in CML that is hosting the free **tacplus** software. If you click the TACSRV icon and select the CONFIG tab, you can view the `tac_plus.conf` file which includes the following important parameters:

- TACACS+ shared secret key: Cisco123!
- Admin user credentials: tacadmin / admin123 / privilege level 15
- Network operator credentials: tacoper / oper123 / privilege level 1
- TCP Port 49

Step 1

The first step in configuring AAA with TACACS+ on R1 is to define the connection parameters of the TACACS+ server.

```
R1(config)# tacacs server TACSRV
R1(config-server-tacacs)# address ipv4 192.168.1.20
R1(config-server-tacacs)# key Cisco123!
R1(config-server-tacacs)# single-connection
R1(config-server-tacacs)# exit
```

NOTE: You will receive a warning message that the option to use a cleartext password will be deprecated in the future. In a production network you would typically want to use an encrypted shared secret key.

The `single-connection` option causes the device to maintain a single TCP connection to the TACACS+ server. This cuts down on connections to the AAA server and can help to speed up the AAA process between the device and the server.

Step2

Next we need to define a TACACS+ server group that will include the TACSRV device configured in Step 1. A server group allows you to include multiple AAA servers for redundancy.

```
R1(config)# aaa group server tacacs+ TACSRVGRP
R1(config-sg-tacacs+)# server name TACSRV
R1(config-sg-tacacs+)# exit
```

Step 3

Let's perform a simple verification step before testing our AAA configuration with an actual SSH login attempt. We will use the `test` command to trigger an authentication attempt for the `tacadmin` user. The `legacy` keyword refers to the default TCP port 49 that TACACS+ uses. If you still have the `debug` commands active, you will see syslog messages confirming a successful authentication.

```
R1# test aaa group TACSRVGRP tacadmin admin123 legacy
Attempting authentication test to server-group TACSRVGRP using tacacs+
```

```
*Nov 28 18:14:25.475: AAA/SG/TEST:Invoked SPI services for PROXY_START and PROXY_STOP
*Nov 28 18:14:25.478: AAA: parse name= idb type=-1 tty=-1
*Nov 28 18:14:25.478: AAA/MEMORY: create_user (0x7F2D5F5BDEF8) user='tacadmin' ruser='NULL' ds0=0 port='' rem_addr='NULL' authen_type=ASCII se:
*Nov 28 18:14:25.478: TAC+: send AUTHEN/START packet ver=192 id=39248394
User was successfully authenticated.
```

```
R1#
*Nov 28 18:14:27.678: TAC+: ver=192 id=39248394 received AUTHEN status = GETPASS
*Nov 28 18:14:27.678: TAC+: send AUTHEN/CONT packet id=39248394
```

```
*Nov 28 18:14:27.878: TAC+: ver=192 id=39248394 received AUTHEN status = PASS
```

```
*Nov 28 18:14:27.881: AAA/MEMORY: free_user (0x7F2D5F5BDEF8) user='tacadmin' ruser=NULL' port='' rem_addr=NULL' authen_type=ASCII service=LO
```

Step 4

Next we will overwrite the current REMOTE-ACCESS authentication and authorization method lists and update them to first use the newly defined TACACS+ server group, and use the local database second if the server is unresponsive.

```
R1(config)# aaa authentication login REMOTE-ACCESS group TACSRVGRP local
R1(config)# aaa authorization exec REMOTE-ACCESS group TACSRVGRP local
```

Step 5

Finally, we will enable some accounting when a user enters (starts) or exits (stops) the EXEC shell, and when a level 1 or level 15 command is entered at the EXEC shell. Accounting also needs to be applied to the VTY lines for EXEC access and the commands entered.

Let's also enable AAA accounting debugging. We should now have all AAA debugging commands active.

```
R1(config)# aaa accounting exec REMOTE-ACCESS start-stop group TACSRVGRP
R1(config)# aaa accounting commands 1 REMOTE-ACCESS start-stop group TACSRVGRP
R1(config)# aaa accounting commands 15 REMOTE-ACCESS start-stop group TACSRVGRP
R1(config)# line vty 0 4
R1(config-line)# accounting exec REMOTE-ACCESS
R1(config-line)# accounting commands 1 REMOTE-ACCESS
R1(config-line)# accounting commands 15 REMOTE-ACCESS
R1(config-line)# end
R1# debug aaa accounting
```

Step 6

Now return to the PC1 console and establish an SSH session with R1 using the `tacoper` username. Notice that this user receives level 1 user EXEC access.

```
PC1:~$ ssh tacoper@10.1.1.1
(tacoper@10.1.1.1) Password: (oper123)
R1> exit
Connection to 10.1.1.1 closed by remote host.
Connection to 10.1.1.1 closed.
```

Next, establish an SSH sessions with R1 using the `tacadmin` username. Notice that the `tacadmin` user receives level 15 privileged EXEC access.

```
PC1:~$ ssh tacadmin@10.1.1.1
(tacadmin@10.1.1.1) Password: (admin123)
R1#
```

Keep this SSH session active but return to the R1 console You should see syslog messages confirming that `tacadmin` was successfully authenticated and authorized to `priv-lvl 15`. You should also see accounting messages indicating the start of an EXEC session.

Step 7

In this final verification step, you will run a few commands `show` commands from the SSH session. This will trigger some accounting activity. You will then investigate the accounting logs on the TACACS+ server.

From the PC1 SSH session to R1, execute the `show ip int brief` and `show run` commands.

Right-click the **TACSRV** icon in CML, select **CONSOLE** and then click the **OPEN CONSOLE** button. If you scroll up through the output, you should see a connection attempt from 192.168.1.1, a successful authentication of the `tacadmin` user, and a successful level 15 authorization.

```
connect from 192.168.1.1 [192.168.1.1]
login query for 'tacadmin' port tty2 from 192.168.1.1 accepted
connect from 192.168.1.1 [192.168.1.1]
Start authorization request
do_author: user='tacadmin'
user 'tacadmin' found
exec authorization request for tacadmin
exec is explicitly permitted by line 11
nas:service=shell (passed thru)
nas:cmd* (passed thru)
nas:absent, server:priv-lvl=15 -> add priv-lvl=15 (k)
added 1 args
out_args[0] = service=shell input copy discarded
out_args[1] = cmd* input copy discarded
```

```
out_args[2] = priv-lvl=15 compacted to out_args[0]
1 output args
authorization query for 'tacadmin' tty2 from 192.168.1.1 accepted
connect from 192.168.1.1 [192.168.1.1]
```

More interestingly, if you right-click the **TACSRV** tab and select the Change Serial option, you can access the server's CLI on Serial Line 1. Press Enter to get the device prompt and type the following command to open the accounting log:

```
tail -f /var/log/tacplus-acct.log
```

The last two entries in the log file should represent the two commands you typed earlier:

```
Nov 24 02:41:31 192.168.1.1      tacadmin      tty2      10.1.1.10      stop      task_id=28      timezone=UTC      service=shell      priv-lvl=1 cmd:
Nov 24 02:41:33 192.168.1.1      tacadmin      tty2      10.1.1.10      stop      task_id=30      timezone=UTC      service=shell      priv-lvl=15cmd:
```

Notice that the `show ip int brief` command is `priv-lvl 1`, while the `show running-config` command is `priv-lvl 15`.

Part 4: Configure server-based AAA using RADIUS

In this last part of the lab, we'll go over the differences between RADIUS and TACACS+, and only describe how to configure it since the RADIUS server in CML does not currently work correctly.

As you already know, TACACS+ uses TCP (port 49), which provides reliable delivery. On the other hand RADIUS uses UDP (ports 1812 for auth, 1813 for accounting).

TACACS+ encrypts the entire body of the packet, while RADIUS only encrypts the password in the access-request packet, leaving the rest of the packet vulnerable.

TACACS+ separates authentication, authorization, and accounting into distinct processes, giving you more granular control. RADIUS combines authentication and authorization - you can't authenticate without also receiving authorization information.

Finally, RADIUS is standardized in [RFC 2865](#) and widely supported across many vendors, while TACACS+ was originally a Cisco proprietary protocol but is now defined in [RFC 8907](#) and supported across multiple vendors.

When it comes to configuring RADIUS, the steps are identical to what we did for TACACS+ except that you need to use the commands to define a RADIUS server, RADIUS server group, and a method list that references the RADIUS server group, as follows:

```
R1(config)# radius server RADSRV
R1(config-radius-server)# address ipv4 192.168.1.20
R1(config-radius-server)# key Cisco123!
R1(config)# exit
R1(config)# aaa group server radius RADSRVGRP
R1(config-sg-radius)# server name RADSRV
R1(config-sg-radius)# exit
R1(config)# aaa authentication login REMOTE-ACCESS group RADSRVGRP local
```

Congratulations! You have completed the lab! You learned all the ways AAA can be deployed on a Cisco router, from using a local database, a TACACS+ server, and a RADIUS server.